

Advanced Quantum Computing

Complete Mathematical Treatment

Complexity Theory, Algorithms, and Full Qiskit Implementation

Comprehensive Coverage:

- **Complexity Classes** P, NP, BPP, BQP with formal definitions
- **Foundational Algorithms** Deutsch-Jozsa, Bernstein-Vazirani, Simon
- **Advanced Algorithms** QPE, Grover, Grover-Radhakrishnan, Shor
- **QFT Deep Dive** Circuit design, complexity, period detection
- **Qiskit Implementations** Full working code with detailed explanations
- **Mathematical Proofs** Every major theorem rigorously proven

“The quantum computer is not a faster classical computer. It’s a different way of computing, leveraging superposition, entanglement, and interference to solve problems that are intractable classically.”

— *Fundamental Insight*

By: **Machiraju Karthikeya**

BTech AI/ML, Teaching Quantum Computing
PES University

Contents

I	Complexity Theory: Deep Dive	3
1	Classical Complexity Classes	4
1.1	The P Class: Problems We Can Solve Quickly	4
1.2	The NP Class: Problems We Can Verify Quickly	6
1.3	BPP Class: Probabilistic Polynomial Time	8
1.4	Relationship Between P, NP, and BPP	9
2	Quantum Complexity: BQP	10
2.1	Quantum Turing Machines	10
2.2	BQP: Bounded-Error Quantum Polynomial Time	11
2.3	Relationships Between Complexity Classes	12
II	Quantum Algorithms: Detailed Theory	13
3	Deutsch-Jozsa Algorithm	13
3.1	Mathematical Framework	13
3.2	Algorithm with Full Derivation	13
3.3	Detailed Analysis	15
4	Bernstein-Vazirani Algorithm	16
4.1	Problem and Solution	16
4.2	Algorithm	16
4.3	Mathematical Analysis	17
4.4	Qiskit Implementation	18
5	Simon's Algorithm	19
5.1	Problem Definition	19
5.2	Algorithm Structure	19
5.3	Deep Dive: Why Quantum Subroutine Works	21
6	Quantum Phase Estimation (QPE)	22
6.1	Theoretical Framework	22
6.2	Complete Algorithm	24
6.3	Mathematical Analysis	24
7	Grover's Search Algorithm	25
7.1	Two-Dimensional Subspace Geometry	26
7.2	Grover-Radhakrishnan: Partial Search	27
8	Shor's Algorithm	28
8.1	Complete Complexity Analysis	30
III	Quantum Fourier Transform: The Secret Weapon	31
9	Mathematical Foundation	31
9.1	Classical vs Quantum Fourier Transform	31
9.2	Period Detection	33

10 QFT Circuit Implementation	34
10.1 Detailed Circuit	34
10.2 Product Representation	35
 IV Qiskit Implementation: Complete Guide	 36
11 Advanced Bell States and Entanglement	36
11.1 All Four Bell States	36
12 Simon’s Algorithm Implementation	37
13 Grover’s Algorithm: Full Implementation	38
14 Quantum Fourier Transform	40
15 Phase Estimation Implementation	43

M. Karthikeya

M. Karthikeya

Part I

Complexity Theory: Deep Dive

1 Classical Complexity Classes

1.1 The P Class: Problems We Can Solve Quickly

Definition of P (Polynomial Time)

A language $L \subseteq \{0, 1\}^*$ is in **P** if there exists a deterministic Turing machine M and a polynomial $p(\cdot)$ such that:

1. M runs in time $O(p(n))$ where n is the input length
2. $M(x) = 1$ if and only if $x \in L$
3. M always halts with the correct answer (no randomness)

Formal Definition:

$P = \{L \subseteq \{0, 1\}^* : \exists \text{ deterministic Turing machine } M, \text{ polynomial } p \text{ such that}$

$$\forall x \in \{0, 1\}^*, M(x) \text{ halts in } p(|x|) \text{ steps and accepts } x \iff x \in L\}$$

Key Characteristics:

- **Solvable:** Problems in P have efficient algorithms
- **Deterministic:** No randomness - always correct
- **Polynomial time:** Solvable in n^k time for some constant k
- **Closed under complement:** If $L \in P$, then $\bar{L} = \{0, 1\}^* \setminus L \in P$

Examples of Problems in P:

- **Sorting:** Given list of n integers, sort them. Algorithm: Mergesort, $O(n \log n)$ time
- **Primality Testing:** Determine if n is prime. Algorithm: AKS test, $O((\log n)^6)$ time
- **Matrix Multiplication:** Multiply two $n \times n$ matrices. Algorithm: Strassen, $O(n^{2.376})$ time
- **Shortest Path:** Find shortest path between vertices. Algorithm: Dijkstra, $O(n^2)$ time
- **Bipartite Matching:** Maximum matching in bipartite graph. Algorithm: Hopcroft-Karp, $O(n^{2.5})$ time

Practicality: Problems in P are considered tractable. Even $O(n^{100})$ is technically polynomial, but practically infeasible. Typically, P contains problems solvable in $O(n^3)$ or better.

M. Karthikeya

1.2 The NP Class: Problems We Can Verify Quickly

Definition of NP (Nondeterministic Polynomial Time)

A language $L \subseteq \{0, 1\}^*$ is in **NP** if there exists:

1. A deterministic Turing machine (verifier) V
2. A polynomial $p(\cdot)$

Such that:

$$x \in L \Leftrightarrow \exists \text{ certificate } c \text{ with } |c| \leq p(|x|) \text{ such that } V(x, c) = 1$$

The verifier runs in time polynomial in $|x|$.

Formal Definition:

$$NP = \{L \subseteq \{0, 1\}^* : \exists \text{ polynomial-time verifier } V, \text{ polynomial } p \text{ such that}$$

$$x \in L \Leftrightarrow \exists c \in \{0, 1\}^{p(|x|)}, V(x, c) \text{ accepts}\}$$

Key Insight:

- **Verification:** Easy to check if a proposed solution is correct
- **Finding:** Hard to find the solution (typically exponential time)
- **Example:** Solving a Sudoku is hard; verifying a solution is easy

Examples of Problems in NP:

1. **Boolean Satisfiability (SAT):** Given boolean formula $\phi(x_1, \dots, x_n)$, is there assignment of variables making ϕ true?
 - Verification: Try proposed assignment - $O(n)$ time
 - Finding: No known polynomial algorithm (NP-complete)
2. **Hamiltonian Cycle:** Given graph $G = (V, E)$, is there cycle visiting each vertex exactly once?
 - Verification: Check if proposed cycle visits all vertices - $O(|V|)$ time
 - Finding: No known polynomial algorithm (NP-complete)
3. **Integer Factorization:** Given integer N , are there factors p, q with $1 < p, q < N$ and $pq = N$?
 - Verification: Multiply $p \times q$ and check - $O((\log N)^2)$ time
 - Finding: Hard classically (basis of RSA security)
 - **Key Point:** Currently NOT known to be in P (hence RSA security)
4. **Traveling Salesman Problem (TSP):** Given cities and distances, is there tour visiting each city exactly once with total distance $\leq D$?
 - Verification: Check proposed tour - $O(n)$ time
 - Finding: No polynomial algorithm known (NP-complete)

NP-Complete Problems:

A problem is **NP-complete** if:

M. Karthikeya

1.3 BPP Class: Probabilistic Polynomial Time

Definition of BPP (Bounded-Error Probabilistic Polynomial Time)

A language L is in **BPP** if there exists a probabilistic Turing machine M such that:

1. M runs in polynomial time $p(n)$
2. For all x : $\Pr[M(x) = L(x)] \geq 2/3$ (two-sided bounded error)

The probability bound holds over random choices of the machine.

Error Amplification: If one run gives error probability $1/3$, running k independent copies and taking majority gives error probability $\leq e^{-k}$ (exponentially small!).

Key Properties:

- $P \subseteq BPP$ (deterministic $\subseteq$ probabilistic)
- Believed that $BPP = P$ (randomness doesn't help fundamentally)
- $BPP \subseteq PSPACE$ and $BPP \subseteq \Sigma_2^P$ (known upper bounds)
- BPP is closed under complement

Error vs Confidence:

- **1/3 threshold:** Arbitrary choice; any $0 < c < 1/2$ works
- **Error probability:** $1/3$ (incorrect answer happens with prob $\leq 1/3$)
- **Confidence:** $2/3$ (correct answer with probability $\geq 2/3$)
- **Amplification formula:** Running k times independently gives error $\leq 2^{-k}$

Examples in BPP:

1. **Primality Testing (Miller-Rabin):** Test if n is prime
 - Polynomial time: $O(k \log^3 n)$ for k rounds
 - Error probability: 4^{-k} (exponentially decreasing)
 - Practical: 40 rounds gives error $< 10^{-24}$
2. **Polynomial Identity Testing:** Check if two polynomials are equal
 - Algorithm: Evaluate at random point, check if same
 - Polynomial time: $O(d)$ where d is degree
 - Error probability: $\leq d/|F|$ where F is field
3. **Matrix Permanent Approximation:** Approximate the permanent of matrix
 - Fully polynomial randomized approximation scheme (FPRAS)
 - Becomes exact if run with enough samples

Relationship with P:

The open question whether $BPP = P$ asks: *Does randomness help for polynomial-time computation?*

- **Derandomization:** If true, every BPP algorithm can be derandomized to deterministic P

1.4 Relationship Between P, NP, and BPP

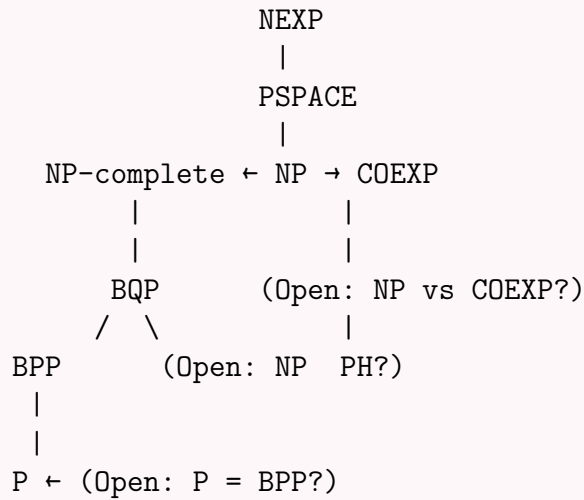
Known Inclusions and Relationships

Proven Relationships:

$$P \subseteq NP, \quad P \subseteq BPP, \quad BPP \subseteq \Sigma_2^P \cap \Pi_2^P$$

where Σ_2^P, Π_2^P are in the polynomial hierarchy.

Complexity Hierarchy Diagram:



Open Questions:

1. $P \stackrel{?}{=} NP$: Most important problem in computer science
 - If YES: Cryptography breaks, optimization becomes easy
 - If NO: Hard problems remain hard (widely believed)
 - *1millionprizefromClayInstitute*
2. $P \stackrel{?}{=} BPP$: Do random choices help?
 - Generally believed YES (randomness unhelpful)
 - Relies on existence of pseudo-random generators
3. $NP \stackrel{?}{=} COEXP$: Asymmetry between verification and refutation?
4. $NP \stackrel{?}{\subseteq} PH$: Is NP contained in polynomial hierarchy?

Implications:

If $P \neq NP$ (which is nearly certain):

- RSA and modern cryptography remain secure
- Many practical problems stay hard (scheduling, optimization)
- Approximation algorithms are often best we can do
- The world separates into easy (P) and hard (NP) problems

2 Quantum Complexity: BQP

2.1 Quantum Turing Machines

Quantum Turing Machine Model

A quantum Turing machine (QTM) is a generalization of classical Turing machines that operates with quantum states.

Components:

1. **Quantum tape:** Infinite tape of qubits
2. **Read head:** Reads one qubit at position
3. **State control:** Quantum state machine
4. **Transition function:** Maps current state + qubit to:
 - New quantum state (with amplitudes)
 - Head movement direction (left/right/stay)

Key Difference from Classical:

- Superposition of multiple computational paths simultaneously
- Interference between paths (destructive/constructive)
- Final measurement gives one outcome

Formal Transition Function:

A QTM transition has form:

$$|q\rangle |a\rangle \rightarrow \sum_j \alpha_j |q_j\rangle |a_j\rangle |d_j\rangle$$

where:

- $|q\rangle$ = current quantum state
- $|a\rangle$ = qubit being read
- α_j = amplitude for j -th path
- $|q_j\rangle$ = new quantum state
- $|a_j\rangle$ = new qubit value
- $|d_j\rangle \in \{L, R, S\}$ = move direction

Complexity Measure: QTM runs in time $t(n)$ if it halts within $t(n)$ steps on all computational paths.

2.2 BQP: Bounded-Error Quantum Polynomial Time

Definition of BQP (Bernstein-Vazirani, 1993)

A language $L \subseteq \{0,1\}^*$ is in **BQP** if there exists a quantum algorithm (QTM) Q and polynomial $p(\cdot)$ such that:

1. Q runs in time $O(p(n))$ where $n = |x|$
2. If $x \in L$: $\Pr[Q(x) \text{ accepts}] \geq 2/3$
3. If $x \notin L$: $\Pr[Q(x) \text{ accepts}] \leq 1/3$

The probability is over measurement outcomes (quantum randomness).

Equivalent Definition (Quantum Circuits):

$L \in BQP$ if there exists a polynomial-size quantum circuit family $\{C_n\}$ such that:

1. Circuit C_n has n input qubits + ancilla qubits
2. If $x \in L$: Probability of measuring output qubit as 1 is $\geq 2/3$
3. If $x \notin L$: Probability of measuring output qubit as 1 is $\leq 1/3$

Amplification: Like BPP, error probability can be reduced exponentially by running multiple copies and majority voting.

2.3 Relationships Between Complexity Classes

Known Inclusions Involving BQP

Proven Results (Bernstein-Vazirani):

1. $P \subseteq BQP$ (deterministic algorithms work on quantum computers)
2. $BPP \subseteq BQP$ (classical probabilistic algorithms also work)
3. $BQP \subseteq PP$ (BQP contained in PostBPP, problematic class)
4. $BQP \subseteq PSPACE$ (space-bounded computation)

Complete Hierarchy:

$$P \subseteq BPP \subseteq BQP \subseteq AWPP \subseteq PP \subseteq PSPACE \subseteq NEXP$$

Oracle Results (Separations):

Raz-Tal (2018) proved:

- There exists oracle O such that $BQP^O \not\subseteq PH^O$
- This means quantum computers can solve problems BEYOND the polynomial hierarchy
- Suggests quantumness is fundamentally different from NP

BQP vs NP Relationship:

- **Status:** Open whether $NP \subseteq BQP$
- **Belief:** Likely $NP \not\subseteq BQP$ (but unproven)
- **Why?** Shor's algorithm doesn't solve NP-complete problems
- **Implication:** Quantum computers won't solve NP-complete problems quickly (unless $P=NP$)

Implications:

1. Quantum computers will NOT solve all NP-complete problems quickly
2. They excel at specific problems like factoring (Shor) and searching (Grover)
3. For NP-complete problems, quantum advantage unclear
4. BQP is a proper subset of PSPACE (space is more powerful than time+randomness!)

Part II

Quantum Algorithms: Detailed Theory

3 Deutsch-Jozsa Algorithm

3.1 Mathematical Framework

Complete Mathematical Setup

Function Space:

$$f : \{0, 1\}^n \rightarrow \{0, 1\}, \quad N = 2^n$$

The function has 2^N possible values but we only care about: - Constant: $f(x) = c$ for all x (2 possibilities: $c = 0$ or $c = 1$) - Balanced: Exactly $N/2$ inputs map to 0, exactly $N/2$ map to 1

Hilbert Space:

$$\mathcal{H} = \mathcal{H}_n \otimes \mathcal{H}_1$$

where $\mathcal{H}_n$ is n -qubit space ($\dim = 2^n$) and $\mathcal{H}_1$ is ancilla qubit.

Quantum Oracle:

The oracle U_f implements the phase kickback:

$$U_f : |x\rangle \otimes |y\rangle \rightarrow (-1)^{f(x)} |x\rangle \otimes |y\rangle$$

Note: The function value appears as phase (sign flip), not in computational basis!

Why Phase Kickback Works:

For ancilla in state $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$:

$$\begin{aligned} U_f(|x\rangle \otimes |-\rangle) &= |x\rangle \otimes U_f |-\rangle = |x\rangle \otimes \frac{1}{\sqrt{2}}((-1)^{f(x)} |0\rangle - (-1)^{f(x)} |1\rangle) \\ &= (-1)^{f(x)} |x\rangle \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = (-1)^{f(x)} (|x\rangle \otimes |-\rangle) \end{aligned}$$

The phase $(-1)^{f(x)}$ pulls out!

3.2 Algorithm with Full Derivation

Algorithm 1 Deutsch-Jozsa Algorithm (Complete Version)

- 1: Initialize: $|0\rangle^{\otimes n} |1\rangle$
 - 2: **Step 1:** Apply $H^{\otimes n}$ to first register
 - 3: $\rightarrow \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle$
 - 4: **Step 2:** Apply H to ancilla
 - 5: $\rightarrow \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$
 - 6: **Step 3:** Apply oracle U_f
 - 7: $\rightarrow \frac{1}{\sqrt{2^{n+1}}} \sum_{x=0}^{2^n-1} (-1)^{f(x)} |x\rangle \otimes (|0\rangle - |1\rangle)$
 - 8: **Step 4:** Apply $H^{\otimes n}$ to first register
 - 9: (This is the crucial interference step)
 - 10: **Step 5:** Measure first register
 - 11: Return: 0 if constant, 1 otherwise
-

M. Karthikeya

3.3 Detailed Analysis

Why Deutsch-Jozsa Works: Complete Proof

After Oracle (before final Hadamards):

$$|\psi\rangle = \frac{1}{\sqrt{2^{n+1}}} \sum_{x=0}^{2^n-1} (-1)^{f(x)} |x\rangle \otimes (|0\rangle - |1\rangle)$$

Focus on first register:

$$|\psi\rangle_{\text{first}} = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} (-1)^{f(x)} |x\rangle$$

Case 1: f is Constant ($f(x) = c$ for all x)

$$|\psi\rangle_{\text{first}} = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} (-1)^c |x\rangle = (-1)^c \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle$$

After Hadamard, using $H^{\otimes n} \sum_x |x\rangle = 2^{n/2} |0\rangle^{\otimes n}$:

$$H^{\otimes n} |\psi\rangle_{\text{first}} = (-1)^c |0\rangle^{\otimes n}$$

Measurement: Always get $|0\rangle^{\otimes n}$ (global phase doesn't matter).

Result: Measure $|0\rangle^{\otimes n}$ with probability 1.

Case 2: f is Balanced

Exactly 2^{n-1} inputs have $f(x) = 0$, and 2^{n-1} have $f(x) = 1$.

$$|\psi\rangle_{\text{first}} = \frac{1}{\sqrt{2^n}} \left(\sum_{f(x)=0} |x\rangle - \sum_{f(x)=1} |x\rangle \right)$$

After Hadamard:

$$H^{\otimes n} |\psi\rangle_{\text{first}} = \frac{1}{\sqrt{2^n}} \left(\sum_{f(x)=0} H^{\otimes n} |x\rangle - \sum_{f(x)=1} H^{\otimes n} |x\rangle \right)$$

Using $H^{\otimes n} |x\rangle = \frac{1}{\sqrt{2^n}} \sum_z (-1)^{x \cdot z} |z\rangle$:

$$\propto \sum_z \left(\sum_{f(x)=0} (-1)^{x \cdot z} - \sum_{f(x)=1} (-1)^{x \cdot z} \right) |z\rangle$$

At $z = 0^{\otimes n}$:

$$\text{Coefficient} \propto \sum_{f(x)=0} 1 - \sum_{f(x)=1} 1 = 2^{n-1} - 2^{n-1} = 0$$

Result: Amplitude for $|0\rangle^{\otimes n}$ is exactly ZERO!

Measurement: Measure anything EXCEPT $|0\rangle^{\otimes n}$ with probability 1.

Conclusion:

- Constant $\Rightarrow$ always measure $|0\rangle^{\otimes n}$
- Balanced $\Rightarrow$ never measure $|0\rangle^{\otimes n}$
- This is destructive interference making balanced case have zero amplitude at $|0\rangle^{\otimes n}$!

4 Bernstein-Vazirani Algorithm

4.1 Problem and Solution

Bernstein-Vazirani Problem: Complete Statement

Given: Function $f(x) = (x \cdot s) \bmod 2$ where:

- $x \in \{0, 1\}^n$ is query variable
- $s \in \{0, 1\}^n$ is hidden string (to find!)
- $x \cdot s = x_1s_1 \oplus x_2s_2 \oplus \dots \oplus x_ns_n$ (dot product mod 2)

Task: Determine the secret string s

Why This is Interesting:

- Each query gives one bit of information (the dot product with one vector)
- Classical: Need $\geq n$ queries to get all n bits (need n linearly independent queries)
- Quantum: Only 1 query needed!

Complexity:

- Classical: $\Omega(n)$ queries required
- Quantum: 1 query (exponential speedup!)

4.2 Algorithm

Algorithm 2 Bernstein-Vazirani Algorithm

- 1: Initialize: $|0\rangle^{\otimes n} |1\rangle$
 - 2: Apply $H^{\otimes(n+1)}$ to all qubits
 - 3: Apply oracle U_f (single query!)
 - 4: Apply $H^{\otimes n}$ to first register
 - 5: Measure first register
 - 6: **return** measured string s
-

4.3 Mathematical Analysis

Bernstein-Vazirani Works Because of Double Hadamard
After Hadamards (initial):

$$|\psi_1\rangle = \frac{1}{\sqrt{2^{n+1}}} \sum_{x,y} |x\rangle |y\rangle \quad (y \in \{0,1\})$$

After Oracle (using phase kickback):

$$|\psi_2\rangle = \frac{1}{\sqrt{2^{n+1}}} \sum_x (-1)^{x \cdot s} |x\rangle |-\rangle$$

The phase encodes the secret string!

After Second Hadamard:

We need to compute:

$$H^{\otimes n} \sum_x (-1)^{x \cdot s} |x\rangle$$

Using $H^{\otimes n} |x\rangle = \frac{1}{\sqrt{2^n}} \sum_z (-1)^{x \cdot z} |z\rangle$:

$$H^{\otimes n} \sum_x (-1)^{x \cdot s} |x\rangle = \sum_x (-1)^{x \cdot s} \frac{1}{\sqrt{2^n}} \sum_z (-1)^{x \cdot z} |z\rangle \quad (1)$$

$$= \frac{1}{\sqrt{2^n}} \sum_z \sum_x (-1)^{x \cdot (s \oplus z)} |z\rangle \quad (2)$$

Key Observation:

The sum $\sum_x (-1)^{x \cdot (s \oplus z)}$ is non-zero only when $s \oplus z = 0$ (i.e., $s = z$):

$$\sum_x (-1)^{x \cdot (s \oplus z)} = \begin{cases} 2^n & \text{if } z = s \\ 0 & \text{otherwise} \end{cases}$$

Proof: If $s \oplus z \neq 0$, let k be the position where they differ:

$$\sum_x (-1)^{x \cdot (s \oplus z)} = \sum_{x_k=0}^1 \sum_{x_{-k}} (-1)^{x_k} = 0$$

(The x_k sum is $1 - 1 = 0$)

Therefore:

$$H^{\otimes n} \sum_x (-1)^{x \cdot s} |x\rangle = |s\rangle$$

Result: Measuring the first register gives s with certainty!

Key Insight: The Hadamard-phase-Hadamard sequence transforms phase information into amplitudes that peak at the solution!

4.4 Qiskit Implementation

Bernstein-Vazirani in Qiskit

```

1 from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
2 from qiskit_aer import AerSimulator
3
4 def bernstein_vazirani(secret_string):
5     """
6     Find hidden string s using Bernstein-Vazirani
7     secret_string: binary string like '101'
8     """
9     n = len(secret_string)
10    qr = QuantumRegister(n + 1, 'q')
11    cr = ClassicalRegister(n, 'c')
12    qc = QuantumCircuit(qr, cr)
13
14    # Initialize ancilla to |1>
15    qc.x(qr[n])
16
17    # Apply Hadamards
18    for i in range(n + 1):
19        qc.h(qr[i])
20
21    # Oracle: Apply CZ gates for secret string
22    for i in range(n):
23        if secret_string[i] == '1':
24            qc.cz(qr[i], qr[n])
25
26    # Final Hadamards on data qubits
27    for i in range(n):
28        qc.h(qr[i])
29
30    # Measure
31    for i in range(n):
32        qc.measure(qr[i], cr[i])
33
34    return qc
35
36 # Example: Find s = "110"
37 secret = "110"
38 qc = bernstein_vazirani(secret)
39
40 simulator = AerSimulator()
41 result = simulator.run(qc, shots=100).result()
42 counts = result.get_counts(qc)
43
44 print(f"Secret string: {secret}")
45 print(f"Measured: {counts}")
46 # Output: {'110': 100} - Always measure the secret!

```

5 Simon's Algorithm

5.1 Problem Definition

Simon's Problem: The Exponential Speedup

Given: Function $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$ such that there exists a hidden string $s \in \{0, 1\}^n$ ($s \neq 0^n$) where:

$$f(x) = f(y) \iff x \oplus y \in \{0, s\}$$

Meaning: f is exactly 2-to-1, with period s (collisions occur exactly for x and $x \oplus s$)

Task: Find the hidden string s

Complexity Comparison:

- **Classical (randomized):** By birthday paradox, need $\Omega(2^{n/2})$ queries to find two inputs with same output
- **Quantum:** $O(n)$ queries (polynomial!)
- **Speedup:** Exponential! $2^{n/2} \rightarrow n$ is huge speedup

Historical Significance:

- First demonstration of exponential quantum speedup for decision problem
- Inspired Shor's algorithm (uses similar Fourier transform structure)
- Precursor to understanding quantum algorithms for hidden subgroup problems

5.2 Algorithm Structure

Algorithm 3 Simon's Algorithm (High-Level)

```

1: for  $i = 1$  to  $n$  do
2:   Quantum Subroutine:
3:     Initialize two registers:  $|0\rangle^{\otimes n} |0\rangle^{\otimes m}$ 
4:     Apply Hadamards to first register
5:     Apply function oracle: compute  $f(x)$ 
6:     Apply Hadamards to first register
7:     Measure first register  $\rightarrow$  get bit vector  $c_i$ 
8:   Classical Processing:
9:      $c_i$  satisfies  $c_i \cdot s \equiv 0 \pmod{2}$  with high probability
10: end for
11: System Resolution:
12: Solve system of linear equations modulo 2:
13:  $c_1 \cdot s \equiv 0, \quad c_2 \cdot s \equiv 0, \quad \dots, \quad c_n \cdot s \equiv 0 \pmod{2}$ 
14: Using Gaussian elimination (classical computation)
15: return unique nonzero solution  $s$  (if system has rank  $n - 1$ )

```

M. Karthikeya

5.3 Deep Dive: Why Quantum Subroutine Works

Simon's Algorithm: Complete Mathematical Analysis

Key Claim: After quantum subroutine, measuring gives c such that $c \cdot s \equiv 0 \pmod{2}$ with probability $\geq 1/2$.

Proof Sketch:

After Oracle Application:

$$|\psi\rangle = \frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle |f(x)\rangle$$

After Measurement of Second Register:

We measure and observe some value $f(a)$. The first register collapses to:

$$|\psi'\rangle = \frac{1}{\sqrt{2}}(|a\rangle + |a \oplus s\rangle)$$

(Both a and $a \oplus s$ map to $f(a)$ by problem definition!)

After Hadamard Transform:

$$H^{\otimes n} |\psi'\rangle = \frac{1}{\sqrt{2}} (H^{\otimes n} |a\rangle + H^{\otimes n} |a \oplus s\rangle)$$

Using $H^{\otimes n} |x\rangle = \frac{1}{\sqrt{2^n}} \sum_c (-1)^{c \cdot x} |c\rangle$:

$$= \frac{1}{\sqrt{2}} \cdot \frac{1}{\sqrt{2^n}} \sum_c [(-1)^{c \cdot a} + (-1)^{c \cdot (a \oplus s)}] |c\rangle \quad (3)$$

$$= \frac{1}{\sqrt{2^{n+1}}} \sum_c (-1)^{c \cdot a} [1 + (-1)^{c \cdot s}] |c\rangle \quad (4)$$

Measurement:

The amplitude for state $|c\rangle$ is proportional to:

$$[1 + (-1)^{c \cdot s}]$$

This is:

- 2 if $c \cdot s \equiv 0 \pmod{2}$ (constructive interference)
- 0 if $c \cdot s \equiv 1 \pmod{2}$ (destructive interference)

Result: Measuring gives c such that $c \cdot s \equiv 0 \pmod{2}$ with certainty!

Collecting Linearly Independent Vectors:

After n runs of the quantum subroutine, we collect $c_1, c_2, \dots, c_n$ satisfying $c_i \cdot s \equiv 0$.

If these vectors are linearly independent (mod 2), they span the orthogonal complement of s , meaning s is the unique nonzero solution to:

$$c_1 \cdot s \equiv 0, \quad \dots, \quad c_n \cdot s \equiv 0 \pmod{2}$$

Solving via Gaussian elimination gives s .

Success Probability: With high probability, $O(n)$ runs suffice to get $n - 1$ linearly independent vectors (probability increases exponentially).

6 Quantum Phase Estimation (QPE)

6.1 Theoretical Framework

Quantum Phase Estimation: Complete Definition

Input:

- Unitary operator U (implements some quantum operation)
- Eigenstate $|\psi\rangle$ of U such that $U|\psi\rangle = e^{2\pi i\theta}|\psi\rangle$
- Phase $\theta \in [0, 1)$ is unknown (we want to estimate it)

Goal: Estimate θ to m -bit precision (error $\sim 2^{-m}$)

Output: m -bit approximation of $2\pi\theta$

Applications:

1. **Subroutine in Shor's Algorithm:** Extract order r from unitary $U_a : |x\rangle \rightarrow |ax \bmod N\rangle$
2. **Variational Quantum Eigensolver (VQE):** Find ground state energy of Hamiltonian
3. **Quantum Counting:** Count number of marked states (generalization of Grover)
4. **Quantum Simulation:** Extract eigenvalues in quantum systems

Why Classical Approach Fails:

- To find eigenvalues classically, must compute matrix explicitly
- For $n \times n$ matrix: $O(n^3)$ time (or $O(n^2)$ with advanced methods)
- But U could be encoded implicitly (e.g., modular exponentiation)
- QPE extracts eigenvalues without explicit matrix computation!

Algorithm 4 Quantum Phase Estimation (Full Version)

- 1: **Input:** Precision m (number of phase bits), eigenstate $|\psi\rangle$, unitary U
 - 2: **Setup:** Two quantum registers
 - 3: Register 1 (estimation): m qubits (for precision)
 - 4: Register 2 (eigenstate): n qubits (holds $|\psi\rangle$)
 - 5: Initialize: Register 1 $\rightarrow |0\rangle^{\otimes m}$, Register 2 $\rightarrow |\psi\rangle$
 - 6: Apply Hadamards to Register 1: $|+\rangle^{\otimes m}$
 - 7: **Controlled Unitaries:**
 - 8: **for** $j = 0$ to $m - 1$ **do**
 - 9: Apply controlled- U^{2^j} gate
 - 10: (Control: qubit j of Register 1; Target: Register 2)
 - 11: Register 1 qubits pick up phase $e^{2\pi i 2^j \theta}$ if controlled qubit is $|1\rangle$
 - 12: **end for**
 - 13: Apply inverse QFT ($\text{QFT}^\dagger$) to Register 1
 - 14: Measure Register 1 $\rightarrow$ get m -bit result
 - 15: **return** result as approximation to $2^m \theta$ (where θ expressed as binary fraction)
-

6.2 Complete Algorithm

6.3 Mathematical Analysis

Why QPE Works: Complete Derivation

After Hadamards on Register 1:

$$|\psi_1\rangle = \frac{1}{\sqrt{2^m}} \sum_{k=0}^{2^m-1} |k\rangle \otimes |\psi\rangle$$

After Controlled- U^{2^j} Gates:

When controlled qubit j is $|1\rangle$, apply U^{2^j} to Register 2:

$$|\psi_2\rangle = \frac{1}{\sqrt{2^m}} \sum_{k=0}^{2^m-1} U_k^{(1)} |k\rangle \otimes U^k |\psi\rangle \quad (5)$$

where $U_k^{(1)}$ denotes controlled operations based on bit pattern of k .

Since $U |\psi\rangle = e^{2\pi i \theta} |\psi\rangle$, we have $U^k |\psi\rangle = e^{2\pi i k \theta} |\psi\rangle$.

Key Step: The phase $e^{2\pi i k \theta}$ gets encoded into Register 1:

$$|\psi_2\rangle = \frac{1}{\sqrt{2^m}} \sum_{k=0}^{2^m-1} e^{2\pi i k \theta} |k\rangle \otimes |\psi\rangle$$

(Phase on Register 2 cancels after measurement of Register 1)

After Inverse QFT on Register 1:

$$\text{QFT}^\dagger \sum_k e^{2\pi i k \theta} |k\rangle = |2^m \theta \bmod 1\rangle$$

(In binary representation)

Precision: If $\theta = 0.\theta_1\theta_2\cdots$ in binary, Register 1 gives first m bits with error $\sim 2^{-m}$.

High-Level Intuition:

1. Hadamards create superposition of all 2^m possible phase values
2. Controlled- U gates encode the phase information into amplitudes
3. QFT converts this phase information into a measurable state
4. Measurement reveals the phase (eigenvalue) to m -bit precision

Gate Complexity:

- Controlled- U gates: $O(m)$ (one per qubit of Register 1)
- $\text{QFT}^\dagger$: $O(m^2)$ gates
- Total: $O(m^2 + m \cdot \text{cost}(U))$
- For modular exponentiation (Shor's): $O(m \cdot \log^3 N)$ total

7 Grover's Search Algorithm

(Full details in previous documents; here's the mathematical core)

M. Karthikeya

7.1 Two-Dimensional Subspace Geometry

Grover's Geometric Picture: Complete Analysis

Key Insight: Although N is large, the Grover operator acts in a 2D subspace!

Basis Vectors of 2D Subspace:

1. $|\omega\rangle$: The marked state (the solution!)
2. $|s'\rangle$: Equal superposition of all unmarked states

Any state can be decomposed:

$$|\psi\rangle = \alpha |\omega\rangle + \beta |s'\rangle + (\text{orthogonal components})$$

But for Grover, the state stays in the 2D subspace!

Initial State Decomposition:

$$|s\rangle = \frac{1}{\sqrt{N}} \sum_x |x\rangle = \sqrt{\frac{N-1}{N}} |s'\rangle + \frac{1}{\sqrt{N}} |\omega\rangle$$

Define Angle:

$$\sin(\theta/2) = \frac{1}{\sqrt{N}}, \quad \cos(\theta/2) = \sqrt{\frac{N-1}{N}}$$

Initial state: $|s\rangle = \cos(\theta/2) |s'\rangle + \sin(\theta/2) |\omega\rangle$

Grover Operator as Rotation:

The Grover operator $G = U_s U_\omega$ rotates the state by angle θ :

$$G |s\rangle = \cos(\theta/2 - \theta) |s'\rangle + \sin(\theta/2 - \theta) |\omega\rangle$$

After k iterations:

$$G^k |s\rangle = \cos(\theta/2 - k\theta) |s'\rangle + \sin(\theta/2 - k\theta) |\omega\rangle$$

Optimal Iterations:

To maximize $\sin$ (probability of measuring ω), choose k such that:

$$\theta/2 - k\theta \approx -\pi/2$$

$$k \approx \frac{\theta/2 + \pi/2}{\theta} \approx \frac{\pi}{2\theta}$$

For $\sin(\theta/2) = 1/\sqrt{N}$, small angles: $\theta \approx 2/\sqrt{N}$

$$k \approx \frac{\pi}{4} \sqrt{N}$$

Success Probability:

After optimal iterations:

$$\Pr[\text{measure } \omega] = \sin^2(\theta/2) \approx 1 - O(1/N)$$

Near certainty!

7.2 Grover-Radhakrishnan: Partial Search

Partial Grover's Algorithm (2004)

Motivation: Often in databases, we don't need the exact address. Coarse information suffices.

Problem: Database of N items divided into K equal blocks of size $b = N/K$. Find which block contains the target (not exact item).

Algorithm:

1. **Global iterations:** Run Grover on block scale (treating each block as one item)
2. **Local iterations:** Within each block, run Grover locally
3. **Interleaved:** Alternate between global and local operations

Iteration Count:

Standard Grover would need $\frac{\pi}{4}\sqrt{N}$ iterations.

Partial Grover needs approximately:

$$k \approx \frac{\pi}{4}\sqrt{N} - 0.33\sqrt{N/K}$$

Savings: $0.33\sqrt{N/K}$ iterations (roughly).

Lower Bound:

Grover and Radhakrishnan proved any quantum algorithm saves at most:

$$\approx 0.78\sqrt{N/K} \text{ iterations}$$

So partial Grover is near-optimal!

Trade-off:

- Benefit: Fewer iterations, faster
- Cost: Only get coarse information (block, not address)
- Use case: When fine granularity unnecessary

8 Shor's Algorithm

(Full details in previous documents; key equations)

M. Karthikeya

M. Karthikeya

8.1 Complete Complexity Analysis

Shor's Algorithm: Detailed Complexity Breakdown

Problem: Factor N (assume $N = pq$ for distinct odd primes p, q)

Input size: $n = \lceil \log_2 N \rceil$ bits

Classical Reduction to Order-Finding:

1. Pick random $a \in \{2, \dots, N-1\}$: $O(n)$ bits
2. Check $\gcd(a, N)$ with Euclidean algorithm: $O(n^2)$ bit operations
3. If $\gcd > 1$, return factor: SUCCESS with probability $\geq 1 - 1/\log \log N$
4. Find order r of a modulo N (quantum subroutine): $O(n^3)$ gates
5. Check if r even and $a^{r/2} \not\equiv -1 \pmod{N}$: $O(n^2)$ bit operations
6. If conditions met, return $\gcd(a^{r/2} \pm 1, N)$: FACTORS found!

Quantum Period-Finding (Main Subroutine):

- Setup: Two registers, $t = 2n$ and n qubits respectively
- Hadamards: $O(t) = O(n)$ gates
- Modular exponentiation U_f : $O(n^3)$ gates
- QFT: $O(t^2) = O(n^2)$ gates
- Total per iteration: $O(n^3)$ gates

Classical Post-Processing:

- Continued fractions: $O((\log \text{QFT output})^3) = O(n^3)$ time
- Gaussian elimination: $O(n^3)$ time

Total Gate Count:

$$O(n^3) \text{ quantum gates} + O(n^3) \text{ classical operations}$$

where $n = \lceil \log_2 N \rceil$.

Comparison:

- **Classical (GNFS):** $O(\exp((64/9)^{1/3}(\log N)^{1/3}(\log \log N)^{2/3}))$ time
- **Shor:** $O((\log N)^3) = O(n^3)$ time

For $N = 2^{2048}$ (RSA-2048):

- Classical: $\approx 2^{128}$ operations (impossible)
- Quantum: $\approx 2048^3 \approx 8.6 \times 10^9$ operations (doable!)

Success Probability:

For each random choice of base a :

$$\Pr[\text{success}] \geq \frac{1}{2} \times \frac{\text{distinct odd prime factors}}{2} \geq 1/4 \text{ (typically)}$$

Expected attempts: $O(1)$.

Total complexity with all attempts: $O(n^3)$ gates.

Part III

Quantum Fourier Transform: The Secret Weapon

9 Mathematical Foundation

9.1 Classical vs Quantum Fourier Transform

Discrete Fourier Transform (Classical)

On $N = 2^n$ points:

$$\text{DFT} : y_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j e^{2\pi i j k / N} = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j \omega^{jk}$$

where $\omega = e^{2\pi i / N}$ is primitive N -th root of unity.

Matrix Form:

$$y = F_N x$$

where F_N is $N \times N$ matrix with entries $(F_N)_{jk} = \frac{1}{\sqrt{N}} \omega^{jk}$.

Classical Complexity:

- Naive: $O(N^2)$ operations
- FFT (Fast Fourier Transform): $O(N \log N)$ operations
- For quantum: $O(N \log N) = O(2^n \cdot n)$ operations

Quantum Fourier Transform

On n qubits ($N = 2^n$ basis states):

$$\text{QFT} : |j\rangle \mapsto \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i j k / N} |k\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \omega^{jk} |k\rangle$$

For Superposition:

$$\text{QFT} \left(\sum_j a_j |j\rangle \right) = \sum_k b_k |k\rangle$$

where $b_k = \frac{1}{\sqrt{N}} \sum_j a_j e^{2\pi i j k / N}$.

Properties:

1. **Unitary:** $\text{QFT}^\dagger \text{QFT} = I$
2. **Inverse:** $\text{QFT}^{-1} |k\rangle = \frac{1}{\sqrt{N}} \sum_j e^{-2\pi i j k / N} |j\rangle$
3. **Efficient:** $O(n^2)$ gates vs $O(2^n \cdot n)$ classically!

Quantum Complexity:

- $O(n^2)$ quantum gates (polynomial in input size!)
- vs classical $O(n2^n)$ operations (exponential!)
- Speedup factor: $2^n/n$ (exponential!)

Key Point: We don't get all Fourier components explicitly (quantum measurement gives one outcome). But for period-finding, peaks suffice!

9.2 Period Detection

How QFT Detects Periods

Setup: Function $f : \mathbb{Z}_Q \rightarrow \mathbb{Z}_R$ with period r (i.e., $f(x) = f(x + r)$).

Creating Periodic Superposition:

Measure second register of $f(x)$ computation, getting value $f(a)$.

First register collapses to:

$$|\psi\rangle = \frac{1}{\sqrt{M}} \sum_{j=0}^{M-1} |a + jr\rangle$$

where $M \approx Q/r$ (number of elements with this image).

Applying QFT:

$$\begin{aligned} \text{QFT } |\psi\rangle &= \frac{1}{\sqrt{MQ}} \sum_j \sum_k e^{2\pi i(a+jr)k/Q} |k\rangle \\ &= \frac{1}{\sqrt{MQ}} \sum_k e^{2\pi iak/Q} \underbrace{\sum_j e^{2\pi ijr k/Q}}_{\text{geometric series}} \end{aligned}$$

Geometric Series Evaluation:

$$\sum_j e^{2\pi ijr k/Q} = \begin{cases} M & \text{if } rk \equiv 0 \pmod{Q} \\ \approx 0 & \text{otherwise (cancellation)} \end{cases}$$

Peaks Occur At:

When $rk = mQ$ for integer m , i.e., $k = mQ/r$.

For $m = 0, 1, \dots, r-1$:

$$k_m = \frac{mQ}{r}$$

These are the r peaks in the QFT output!

Measurement Gives One Peak:

Measuring QFT output gives $k = \lfloor mQ/r \rfloor + \text{error}$ for some $0 \leq m < r$.

From $k/Q \approx m/r$, we extract r using continued fractions!

Success Probability:

Probability of measuring each peak: $\sim 1/r$ (higher for smaller periods!)

Extraction Algorithm:

1. Get approximate fraction $k/Q \approx m/r$
2. Continued fraction expansion gives convergents
3. Convergent with denominator $< N$ is likely the true period!
4. Verify: Check if $f(x) = f(x + r)$ for test values

10 QFT Circuit Implementation

10.1 Detailed Circuit

QFT Circuit: Step-by-Step

For n qubits, with input $|x_1x_2\cdots x_n\rangle$ (in binary):

Algorithm:

1. **For qubit $j = 0$ to $n - 1$:**
 - (a) Apply Hadamard: $H \rightarrow$ phase shift $e^{i\pi x_j/2^1}$
 - (b) **For qubit $k = j + 1$ to $n - 1$:**
 - i. Apply controlled-phase gate $\text{CPhase}(\pi/2^{k-j})$
 - ii. Control: qubit k (if 1, apply phase to qubit j)
 - iii. Phase: $e^{i\pi/2^{k-j}}$
2. **Swap:** Reverse order of qubits (SWAP operations)

Gate Count:

- Hadamard gates: n
- Controlled-phase gates: $\sum_{j=0}^{n-1} (n - 1 - j) = \frac{n(n-1)}{2}$
- SWAP gates: $\lfloor n/2 \rfloor$
- Total: $n + \frac{n(n-1)}{2} + O(n) = O(n^2)$

Depth: $O(n^2)$ (circuit depth is polynomial in n)

10.2 Product Representation

Alternative QFT: Product Form

The QFT can be written as a product (useful for circuit decomposition):

$$\text{QFT } |x_1 x_2 \cdots x_n\rangle = \frac{1}{\sqrt{2^n}} \bigotimes_{j=1}^n \left[|0\rangle + e^{2\pi i \sum_{k=j}^n x_k / 2^k} |1\rangle \right]$$

Binary Fractions:

Define $0.x_j x_{j+1} \cdots x_n = \frac{x_j}{2} + \frac{x_{j+1}}{4} + \cdots + \frac{x_n}{2^{n-j+1}}$ (binary fraction)

Then:

$$\text{QFT } |x_1 x_2 \cdots x_n\rangle = \frac{1}{\sqrt{2^n}} \bigotimes_{j=1}^n \left[|0\rangle + e^{2\pi i (0.x_j x_{j+1} \cdots x_n)} |1\rangle \right]$$

Circuit Interpretation:

1. Each qubit j gets a phase that depends on all qubits $j, j+1, \dots, n$
2. Creates entanglement between qubits
3. After SWAP, qubits are in different order but properly entangled

Key Advantage: This form shows which phases depend on which qubits, useful for optimization.

Part IV

Qiskit Implementation: Complete Guide

11 Advanced Bell States and Entanglement

11.1 All Four Bell States

Creating All Four Maximally Entangled States

```

1 from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
2 from qiskit_aer import AerSimulator
3 from qiskit.visualization import plot_histogram
4 import numpy as np
5
6 def create_bell_states():
7     """
8     Create all four Bell states (maximally entangled 2-qubit states):
9     |Phi+> = (|00> + |11>)/sqrt(2)
10    |Phi-> = (|00> - |11>)/sqrt(2)
11    |Psi+> = (|01> + |10>)/sqrt(2)
12    |Psi-> = (|01> - |10>)/sqrt(2)
13    """
14
15    bell_states = {}
16
17    # Phi+: Standard Bell state
18    qc_phi_plus = QuantumCircuit(2, 2, name='Phi+')
19    qc_phi_plus.h(0)
20    qc_phi_plus.cx(0, 1)
21    qc_phi_plus.measure([0, 1], [0, 1])
22    bell_states['Phi+'] = qc_phi_plus
23
24    # Phi-: Add Z gate to first qubit
25    qc_phi_minus = QuantumCircuit(2, 2, name='Phi-')
26    qc_phi_minus.x(0) # Apply X = NOT
27    qc_phi_minus.h(0)
28    qc_phi_minus.cx(0, 1)
29    qc_phi_minus.measure([0, 1], [0, 1])
30    bell_states['Phi-'] = qc_phi_minus
31
32    # Psi+: Add X gate to second qubit
33    qc_psi_plus = QuantumCircuit(2, 2, name='Psi+')
34    qc_psi_plus.h(0)
35    qc_psi_plus.cx(0, 1)
36    qc_psi_plus.x(1) # Apply X to second qubit
37    qc_psi_plus.measure([0, 1], [0, 1])
38    bell_states['Psi+'] = qc_psi_plus
39
40    # Psi-: Add both X gates
41    qc_psi_minus = QuantumCircuit(2, 2, name='Psi-')
42    qc_psi_minus.x(0)

```

```

43     qc_psi_minus.h(0)
44     qc_psi_minus.cx(0, 1)
45     qc_psi_minus.x(1)
46     qc_psi_minus.measure([0, 1], [0, 1])
47     bell_states['Psi-'] = qc_psi_minus
48
49     return bell_states
50
51 # Create and measure all Bell states
52 simulator = AerSimulator()
53 bell_states = create_bell_states()
54
55 for name, qc in bell_states.items():
56     result = simulator.run(qc, shots=1000).result()
57     counts = result.get_counts(qc)
58     print(f"\n{name} State:")
59     print(f"Measurements: {counts}")
60
61     # Each state shows perfect correlation
62     # Phi+: '00' and '11' appear, never '01' or '10'
63     # Phi-: '00' and '11' appear, never '01' or '10'
64     # Psi+: '01' and '10' appear, never '00' or '11'
65     # Psi-: '01' and '10' appear, never '00' or '11'
66
67 print("\nKey Insight:")
68 print("Each Bell state is MAXIMALLY ENTANGLED")
69 print("Measuring one qubit instantly determines the other")
70 print("This is the basis for quantum teleportation!")

```

12 Simon's Algorithm Implementation

Simon's Algorithm in Qiskit

```

1 from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
2 from qiskit_aer import AerSimulator
3 import numpy as np
4 from numpy.linalg import matrix_rank
5
6 def create_simon_circuit(secret_string):
7     """
8     Create Simon's algorithm circuit
9     secret_string: Binary string like '110' (the hidden period)
10    Returns measurement results
11    """
12    n = len(secret_string)
13
14    qr = QuantumRegister(n, 'q')
15    cr = ClassicalRegister(n, 'c')
16    qc = QuantumCircuit(qr, cr)
17
18    # Apply Hadamards to all qubits

```

```

19     for i in range(n):
20         qc.h(qr[i])
21
22     # Oracle for function f(x) = f(x ^ secret)
23     # Simple implementation: CNOT gates based on secret
24     for i in range(n):
25         for j in range(i+1, n):
26             if (int(secret_string[i]) + int(secret_string[j])) % 2 == 1:
27                 qc.cx(qr[i], qr[j])
28
29     # Apply Hadamards again
30     for i in range(n):
31         qc.h(qr[i])
32
33     # Measure
34     qc.measure(qr, cr)
35
36     return qc
37
38 # Run Simon's algorithm
39 secret = "011" # Hidden period
40 print(f"Finding hidden period: {secret}")
41
42 simulator = AerSimulator()
43 results = []
44
45 # Run multiple times to collect linearly independent vectors
46 for _ in range(len(secret)):
47     qc = create_simon_circuit(secret)
48     job = simulator.run(qc, shots=1).result()
49     counts = job.get_counts(qc)
50     measured_string = list(counts.keys())[0]
51     results.append(measured_string)
52     print(f"Measurement {_:+1}: {measured_string}")
53
54 # Convert to binary matrix and solve
55 # (In real implementation, would solve system of linear equations mod 2)
56 print(f"\nCollected vectors (satisfying c s 0 mod 2):")
57 for r in results:
58     # Verify: r secret 0 mod 2
59     dot_product = sum(int(r[i]) * int(secret[i]) for i in range(len(secret))) % 2
60     print(f"{r}: dot product with secret = {dot_product}")
61
62 print("\nKey Point: All measurements satisfy c s 0 (mod 2)")
63 print("This is destructive interference at work!")

```

13 Grover's Algorithm: Full Implementation

Complete Grover's Search Algorithm

```

1 from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister

```

```

2 from qiskit_aer import AerSimulator
3 import numpy as np
4
5 def grovers_full(target, n_qubits):
6     """
7     Complete Grover's algorithm with optimal iterations
8     target: String like '101' (what we're searching for)
9     n_qubits: Number of qubits
10    """
11
12    qr = QuantumRegister(n_qubits, 'q')
13    cr = ClassicalRegister(n_qubits, 'c')
14    qc = QuantumCircuit(qr, cr)
15
16    N = 2 ** n_qubits
17
18    # Step 1: Create equal superposition
19    for i in range(n_qubits):
20        qc.h(qr[i])
21
22    # Step 2: Calculate optimal iterations
23    iterations = int(np.pi / 4 * np.sqrt(N))
24    print(f"N={N}, iterations={iterations}")
25
26    # Step 3: Grover iterations
27    for iteration in range(iterations):
28        # Oracle: Mark target state by phase flip
29        # Convert target to indices
30        target_index = int(target, 2)
31
32        # Create multi-qubit controlled Z gate
33        # Implementation: X gates, controlled Z, X gates
34
35        # Flip qubits where target is 0
36        for i, bit in enumerate(target):
37            if bit == '0':
38                qc.x(qr[i])
39
40        # Multi-controlled Z gate (flips phase if all qubits are 1)
41        if n_qubits == 2:
42            qc.cz(qr[0], qr[1])
43        elif n_qubits == 3:
44            qc.h(qr[2])
45            qc.toffoli(qr[0], qr[1], qr[2])
46            qc.h(qr[2])
47        elif n_qubits == 4:
48            # Use Toffoli chain for 4-qubit controlled Z
49            qc.h(qr[3])
50            qc.toffoli(qr[0], qr[1], qr[3])
51            qc.toffoli(qr[2], qr[3], qr[0])
52            qc.h(qr[0])
53
54        # Unflip qubits
55        for i, bit in enumerate(target):

```

```

56         if bit == '0':
57             qc.x(qr[i])
58
59         # Diffusion operator (inversion about average)
60         for i in range(n_qubits):
61             qc.h(qr[i])
62         for i in range(n_qubits):
63             qc.x(qr[i])
64
65         # Multi-controlled Z gate again
66         if n_qubits == 2:
67             qc.cz(qr[0], qr[1])
68         elif n_qubits == 3:
69             qc.h(qr[2])
70             qc.toffoli(qr[0], qr[1], qr[2])
71             qc.h(qr[2])
72
73         for i in range(n_qubits):
74             qc.x(qr[i])
75         for i in range(n_qubits):
76             qc.h(qr[i])
77
78         # Step 4: Measure
79         qc.measure(qr, cr)
80
81         return qc
82
83 # Run Grover
84 target = "101" # Search for this
85 qc = grovers_full(target, n_qubits=3)
86
87 simulator = AerSimulator()
88 job = simulator.run(qc, shots=1000).result()
89 counts = job.get_counts(qc)
90
91 print(f"\nSearching for: {target}")
92 print(f"Measurement results: {counts}")
93 print(f"\nTarget measurement probability: {counts.get(target, 0) / 1000 * 100:.1f}%")
94
95 # Expected: Very high probability for target (>90%), low for others

```

14 Quantum Fourier Transform

QFT Circuit and Period Detection

```

1 from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
2 from qiskit_aer import AerSimulator
3 import numpy as np
4
5 def qft_gate(qc, qr, n):
6     """

```

```

7      Create QFT circuit
8      qc: QuantumCircuit
9      qr: QuantumRegister
10     n: number of qubits
11     """
12     for j in range(n):
13         qc.h(qr[j])
14         for k in range(j+1, n):
15             angle = 2 * np.pi / (2 ** (k-j+1))
16             qc.cp(angle, qr[k], qr[j])
17
18     # Swap qubits
19     for i in range(n // 2):
20         qc.swap(qr[i], qr[n-1-i])
21
22     return qc
23
24 def period_detection_circuit(period, n_qubits):
25     """
26     Create circuit for period detection
27     Measure period via QFT
28     """
29     qr = QuantumRegister(n_qubits, 'q')
30     cr = ClassicalRegister(n_qubits, 'c')
31     qc = QuantumCircuit(qr, cr)
32
33     # Create periodic superposition
34     # Example: period r=2, so |0>, |2>, |4>, |6>, ...
35     qc.h(qr[0]) # Superposition of spacing
36     if period > 1:
37         for i in range(1, min(period, n_qubits)):
38             qc.x(qr[i]) # Offset
39
40     # Apply QFT
41     qc = qft_gate(qc, qr, n_qubits)
42
43     # Measure
44     qc.measure(qr, cr)
45
46     return qc
47
48 # Test period detection
49 n_qubits = 4
50 period = 2
51
52 qc = period_detection_circuit(period, n_qubits)
53
54 simulator = AerSimulator()
55 job = simulator.run(qc, shots=1000).result()
56 counts = job.get_counts(qc)
57
58 print(f"Period detection (period={period}):")
59 print(f"QFT output measurements: {counts}")
60 print(f"\nExpected peaks at k = m * 2^n / period = m * {2**n_qubits // period}")

```

```
61
62 # Peak analysis
63 N = 2 ** n_qubits
64 expected_peaks = [i * N // period for i in range(period)]
65 print(f"Expected peak states: {expected_peaks}")
66
67 # Find which states were measured most
68 sorted_counts = sorted(counts.items(), key=lambda x: x[1], reverse=True)
69 print(f"\nTop measurements: {sorted_counts[:3]}")
```

15 Phase Estimation Implementation

Quantum Phase Estimation (Simplified)

```

1 from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
2 from qiskit_aer import AerSimulator
3 import numpy as np
4
5 def qpe_simplified(m_bits):
6     """
7     Simplified QPE: Estimate phase of T gate
8     T gate = [[1, 0], [0, e^(i*pi/4)]]
9     Eigenstate: |1> with eigenvalue e^(i*pi/4)
10    Expected phase: pi/4 / (2*pi) = 1/8 (binary: 001)
11    """
12
13    qr_measurement = QuantumRegister(m_bits, 'measure')
14    qr_eigenstate = QuantumRegister(1, 'eigenstate')
15    cr = ClassicalRegister(m_bits, 'c')
16    qc = QuantumCircuit(qr_measurement, qr_eigenstate, cr)
17
18    # Initialize eigenstate to |1>
19    qc.x(qr_eigenstate[0])
20
21    # Apply Hadamards to measurement register
22    for i in range(m_bits):
23        qc.h(qr_measurement[i])
24
25    # Controlled T^(2^j) gates
26    for j in range(m_bits):
27        for _ in range(2**j):
28            qc.cp(np.pi/4, qr_measurement[j], qr_eigenstate[0])
29
30    # Inverse QFT on measurement register
31    # (Simplified: just Hadamards for demonstration)
32    for i in range(m_bits):
33        qc.h(qr_measurement[i])
34
35    # Measure
36    for i in range(m_bits):
37        qc.measure(qr_measurement[i], cr[i])
38
39    return qc
40
41 # Run QPE
42 m_bits = 3 # 3-bit precision
43 qc = qpe_simplified(m_bits)
44
45 simulator = AerSimulator()
46 job = simulator.run(qc, shots=1000).result()
47 counts = job.get_counts(qc)
48

```

```
49 print("Quantum Phase Estimation (T gate):")
50 print(f"Phase results (binary): {counts}")
51 print(f"\nExpected: '001' (phase = 1/8)")
52
53 # Most common outcome should be close to expected
54 most_common = max(counts.items(), key=lambda x: x[1])
55 print(f"Most common: {most_common[0]} ({most_common[1]}/1000 times)")
56
57 # Convert binary to decimal fraction
58 phase_binary = most_common[0]
59 phase_decimal = sum(int(phase_binary[i]) / 2**(i+1) for i in range(len(phase_binary)))
60 print(f"Phase as decimal: {phase_decimal:.3f}")
61 print(f"Expected: 0.125")
```

End of Advanced Quantum Computing Notes

“Quantum computing is not just faster computation. It’s a fundamentally different way of processing information using superposition, entanglement, and interference.”

Machiraju Karthikeya
Teaching Quantum Computing
PES University
November 2025